UET.CS2043 - Lập trình nâng cao

Tích hợp và Triển khai

Nội dung bài học

Hiểu quy trình phát triển phần mềm hiện đại

Nắm vững vòng đời xây dựng phần mềm

Hiểu tầm quan trọng của Code Quality và Automation

Sử dụng thành thạo Maven quản lý dự án

Triển khai Logging chuẩn với SLF4J/Logback

Viết Script CI cơ bản với GitHub Actions

Dự án trên lớp vs. dự án thực tế

Tiêu chí	Dự án trên lớp	Doanh nghiệp
Thư viện	Tải file .jar, copy vào folder ‘lib’, add path thủ công.	Khai báo trong 'pom.xml', tự động tải
Build	Bấm "Run" trong IDE (Eclipse/Netbeans).	Chạy lệnh CLI ('mvn package'), độc lập IDE
Debug	System.out.println("Test 123");	Logging System (File, Database, Levels)
Deploy	Copy paste code qua USB/Zalo.	CI/CD Pipeline, Docker

Vấn đề: "It works on my computer"

- Tại sao code **chạy ngon** trên máy bạn nhưng **99+ errors** trên máy thầy?
- Môi trường khác nhau:
  - JDK 8 vs JDK 17,
  - Windows vs Linux.
- Hard-coded paths: C:\Users\Tuan\Documents\data.txt
- Thiếu thư viện: Quên copy file .jar kèm theo.
- Xung đột phiên bản: Bạn dùng v1.0, máy thầy có v2.0.

Giải pháp: Automation

Configuration as Code

Build Tool
Maven/Gradle

Code Quality
Checkstyle/Sonar

Logging
Logback/SLF4J

CI/CD
GitHub Actions

Apache Maven

Quản lý dự án và tự động hóa việc build chương trình

Maven là gì?

- Là công cụ quản lý dự án phần mềm chủ yếu cho Java
- Dựa trên khái niệm Project Object Model (POM)
- Chức năng chính:
  - Quản lý thư viện phụ thuộc (Dependencies)
  - Biên dịch, đóng gói, chạy test, tạo tài liệu
  - Quản lý phiên bản (Release management)

```
• project-root/
• ├── pom.xml (File cấu hình chính)
• ├── src/
• │ ├── main/
• │ │ ├── java/ (Source code)
• │ │ └── resources/ (Configs, images)
• │ └── test/
• │ ├── java/ (Unit Test code)
• │ └── resources/ (Test configs)
• └── target/ (Output: .class, .jar)
```

POM.xml

Project Object Model chứa thông tin định danh dự án

```
<project>
    <modelVersion>4.0.0</modelVersion>

    <!-- GAV Coordinates -->
    <groupId>com.example</groupId>       <!-- Tên tổ chức -->
    <artifactId>my-app</artifactId>    <!-- Tên dự án -->
    <version>1.0-SNAPSHOT</version>    <!-- Phiên bản -->

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
    </properties>
</project>
```

Quản lý thư viện

- Không cần download file .jar, chỉ cần khai báo thông tin trong thẻ <dependencies>
- Maven sẽ tự động tìm và tải từ Maven Central Repository

```
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>5.9.0</version>
    <scope>test</scope>
</dependency>
```

Scope	Mô tả	Ví dụ
compile	(Mặc định) Có mặt ở tất cả các giai đoạn (build, test, run).	Spring Boot, Commons-Lang
test	Chỉ dùng khi compile và chạy test. Không có trong file .jar cuối cùng.	JUnit, Mockito
provided	Cần khi biên dịch, nhưng khi chạy thì môi trường (Server) sẽ cung cấp.	Servlet API (Tomcat đã có sẵn)
runtime	Không cần khi biên dịch, chỉ cần khi chạy.	JDBC Driver (MySQL Driver)

Repositories

Maven tìm thư viện ở đâu?

1. **Local Repository**: Thư mục lưu local trên máy tính (thường ở ~/.m2/repository)
   - Maven tìm ở đây trước
2. **Central Repository**: Kho lưu trữ online (repo.maven.apache.org)
   - Nếu Local không có, Maven lên đây tải
3. **Remote/Private Repository**: Kho riêng của công ty (Nexus, Artifactory) để chứa các thư viện nội bộ không công khai

Maven Build Lifecycle

Quy trình build gồm các phases:

validate	compile	test	package	install	deploy
Kiểm tra cấu trúc	Dịch code	Chạy Unit Test	Đóng gói (JAR)	Lưu vào Local	Up lên Remote

**Chạy phase sau sẽ tự động kích hoạt tất cả phase trước nó*

Các lệnh Maven thường dùng

mvn clean

Xóa thư mục target để build lại từ đầu

mvn compile

Chỉ biên dịch mã nguồn trong src/main/java

mvn test

Biên dịch và chạy các test case. Tạo báo cáo trong target/surefire-reports

mvn package

Chạy test, nếu pass thì đóng gói thành JAR/WAR

mvn install

Đóng gói và đưa vào Local Repository (để các project khác trong máy dùng được)

Maven Plugins

- Bản thân Maven chỉ là một cái khung. Mọi công việc thực tế do các Plugin đảm nhận
  - **maven-compiler-plugin**: Dùng để biên dịch code (Java 8, 11, 17...)
  - **maven-surefire-plugin**: Dùng để chạy Unit Test
  - **maven-jar-plugin**: Tạo file JAR, cấu hình Manifest (Main-Class)
  - **maven-checkstyle-plugin**: Kiểm tra code standard (sẽ học sau)

Quiz: Maven Lifecycle

Câu hỏi: Nếu bạn chạy lệnh mvn package, và Unit Test bị lỗi (Fail), điều gì xảy ra?

A. Vẫn tạo file .jar

Maven bỏ qua lỗi và tiếp tục đóng gói

B. Dừng ngay lập tức

Không có file .jar nào được tạo ra.

Quiz: Dependency Scope

Câu hỏi: Thư viện JUnit nên để scope là gì?

A. compile

B. test

C. runtime

D. provided

Logging Strategy

Tại sao cần Logging?

- **Debugging**: Tìm nguyên nhân lỗi trong quá trình phát triển
- **Auditing**: Theo dõi hành vi người dùng
  - Ai đã xóa bản ghi này? Lúc mấy giờ?
- **Monitoring**: Theo dõi sức khỏe hệ thống
  - Thời gian phản hồi, Tải server
- **Forensics**: Điều tra sự cố sau khi đã xảy ra

System.out.println vs Logger

System.out.println

- Luôn luôn chạy, không thể tắt.
- Chỉ in ra Console (mất khi tắt app).
- Đơn luồng, làm chậm ứng dụng (Blocking I/O).
- Không có thông tin ngữ cảnh (Timestamp, Class, Thread).

Logging Framework

- Luôn luôn chạy, không thể tắt.
- Chỉ in ra Console (mất khi tắt app).
- Đơn luồng, làm chậm ứng dụng (Blocking I/O).
- Không có thông tin ngữ cảnh (Timestamp, Class, Thread).

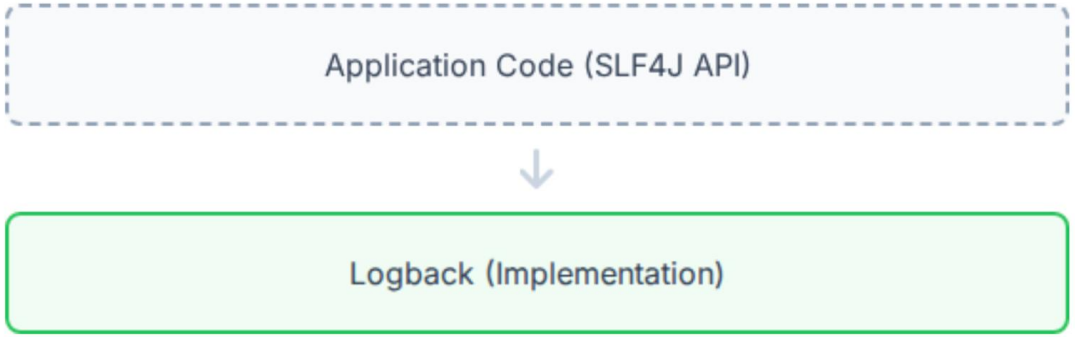
Kiến trúc: SLF4J & Logback

- **SLF4J (Simple Logging Facade for Java)**: Là một interface, dev chỉ code với SLF4J
- **Logback / Log4j2**: Là thư viện cài đặt thực tế (Implementation), chịu trách nhiệm ghi log

Thứ tự mức độ nghiêm trọng tăng dần:

- **TRACE**: Chi tiết nhỏ nhất (vào hàm, ra hàm, giá trị biến tạm)
- **DEBUG**: Thông tin để gỡ lỗi. Chỉ dùng khi Develop
- **INFO**: Thông tin vận hành bình thường (Start app, Login success)
- **WARN**: Cảnh báo, hệ thống vẫn chạy nhưng có rủi ro (Disk sắp đầy, Input sai định dạng)
- **ERROR**: Lỗi chức năng, Exception. Cần chú ý

Quy tắc: Nếu cấu hình Level là INFO, thì TRACE và DEBUG sẽ bị ẩn đi



Parameterized Logging

Hiệu năng String Concatenation

Cách sai: Cộng chuỗi ngay cả khi log bị tắt

```
// Tốn chi phí cộng chuỗi dù log level là INFO
logger.debug("Giá trị là: " + obj.toString() + " user: " + user.getName());
```

Cách đúng: Dùng placeholder {}

```
// Chỉ thực hiện toString() khi debug được bật
logger.debug("Giá trị là: {} user: {}", obj, user.getName());
```

Demo Cấu hình Logback

```
<configuration>
    <appender name="FILE" class="ch.qos.logback.core.FileAppender">
        <file>app.log</file>
        <encoder>
            <pattern>%d{HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>

    <root level="info">
        <appender-ref ref="FILE" />
    </root>
</configuration>
```

Demo Cấu hình Logback

Nếu cấu hình `<root level="WARN">`, thì những dòng log nào sau đây sẽ xuất hiện?

```
logger.trace("Trace message");
logger.debug("Debug message");
logger.info("Info message");
logger.warn("Warn message");
logger.error("Error message");
```

Static Code Analysis

Checkstyle & Code Quality

Phân tích tĩnh là gì?

- Là quá trình kiểm tra mã nguồn tự động mà **không cần chạy chương trình**
- Mục tiêu:
  - Tìm lỗi tiềm ẩn (Potential Bugs) - NullPointer, Resource Leak
  - Kiểm tra phong cách code có theo chuẩn không (Coding Standards) - Checkstyle
  - Đo độ phức tạp (Complexity) - Cyclomatic Complexity
  - Tìm mã trùng lặp (Duplication)

Giới thiệu Checkstyle

- Checkstyle tập trung chủ yếu vào Format và Coding Standard

Các lỗi thường bắt:

- Tên biến/hàm sai quy tắc (CamelCase)
- Thiếu Javadoc cho public API
- Dòng code quá dài (> 120 ký tự)
- Import thư viện không dùng đến (Unused imports)
- Nhiều câu lệnh trên 1 dòng

Bộ quy tắc phổ biến:

- Sun Checks: Chuẩn gốc của Java
- Google Checks: Chuẩn của Google (phổ biến hơn)

Tích hợp Maven Checkstyle Plugin

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-checkstyle-plugin</artifactId>
    <version>3.2.0</version>
    <configuration>
        <configLocation>google_checks.xml</configLocation>
        <failOnViolation>true</failOnViolation>
    </configuration>
    <executions>
        <execution>
            <phase>validate</phase>
            <goals><goal>check</goal></goals>
        </execution>
    </executions>
</plugin>
```

Khái niệm: Cyclomatic Complexity

- Độ phức tạp vòng: Đo lường số lượng đường đi độc lập qua mã nguồn (số lượng if/else, for, switch, case)
- Nếu hàm có quá nhiều if/else lồng nhau

➔ Complexity cao

➔ Khó test, khó bảo trì

- Checkstyle có thể cảnh báo nếu một hàm có độ phức tạp > 10

➔ Giải pháp: Tách hàm nhỏ hơn (Refactoring)

Demo: Đọc Checkstyle report

Khi chạy mvn checkstyle:check

```
[ERROR] src/main/java/App.java:10:5: Method name 'TinhTong' must match pattern '^[a-z][a-zA-Z0-9]*$'. [MethodName]
[ERROR] src/main/java/App.java:15: Line is longer than 100 characters. [LineLength]
```

➔ Phải sửa hết lỗi này mới Build thành công

Các công cụ khác (Optional)

- Ngoài Checkstyle, còn có:
  - **SpotBugs (FindBugs):** Tìm lỗi logic (ví dụ: so sánh String bằng `==`, quên đóng File Stream)
  - **PMD:** Tìm code thừa, code chết, tối ưu code
  - **SonarQube:** Nền tảng tổng hợp tất cả các chỉ số trên, hiển thị Dashboard quản lý chất lượng dự án

CI/CD Automation

GitHub Actions Workflow

CI/CD là gì?

- **CI (Continuous Integration)**
  - Tích hợp liên tục. Tự động Build và Test mỗi khi Code được đẩy lên Repo chung
  - Mục tiêu: Phát hiện lỗi sớm (Fail Fast)
- **CD (Continuous Delivery)**
  - Triển khai liên tục. Tự động đóng gói và triển khai ứng dụng lên môi trường Staging/Production
  - Mục tiêu: Phát hành phần mềm nhanh chóng

Kiến trúc GitHub Actions

- **Workflow**: Quy trình tự động hóa (định nghĩa bằng file YAML)
- **Event**: Sự kiện kích hoạt (Push, Pull Request, Schedule, Manual)
- **Job**: Tập hợp các bước thực hiện (chạy trên cùng 1 máy ảo - Runner)
- **Step**: Từng hành động nhỏ (Checkout code, Cài Java, Chạy Maven)
- **Runner**: Máy chủ thực thi (Ubuntu-latest, Windows-latest)

Cấu trúc file YAML (.github/workflows)

```yaml
name: Java CI

on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v3  # Lấy code về

    - uses: actions/setup-java@v3 # Cài JDK
      with:
        java-version: '17'
        distribution: 'temurin'

    - run: mvn package # Chạy lệnh build
```

Caching Dependencies

- Maven tải thư viện rất lâu. Có thể Cache lại thư mục ~/.m2, để tăng tốc độ Build trên CI

```
- name: Set up JDK 17 with Cache
  uses: actions/setup-java@v3
  with:
    java-version: '17'
    distribution: 'temurin'
    cache: 'maven'  <-- Chỉ cần thêm dòng này
```

GitHub sẽ tự động lưu cache sau lần build đầu tiên và tải lại ở các lần sau

Workflow Badge

- Là biểu tượng trạng thái hiển thị trên file README.md
- Tạo niềm tin cho người sử dụng

Multi-OS Testing (Matrix Build)

- Làm thế nào đảm bảo code chạy tốt trên cả Windows và Linux?

```
jobs:
  build:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
```

GitHub sẽ tạo ra 3 Job chạy song song trên 3 hệ điều hành khác nhau

Artifacts (Lưu trữ kết quả)

- Sau khi Build xong, máy ảo Runner sẽ bị hủy ➔ File .jar bị mất
- Để giữ lại file .jar (hoặc báo cáo Test) để tải về, dùng Upload Artifacts

```
- name: Upload JAR
  uses: actions/upload-artifact@v3
  with:
    name: my-app-jar
    path: target/*.jar
```

Branch Protection

- Thiết lập trên GitHub Settings:
  - Yêu cầu CI phải xanh (Pass) mới được Merge code vào nhánh main.
  - Ngăn chặn việc Push code trực tiếp vào main (Phải qua Pull Request).

➔ Đây là chốt chặn chất lượng quan trọng nhất của quy trình DevOps

Quiz: CI Failure

Nếu CI báo đỏ (Fail) trên Pull Request của bạn, bạn nên làm gì?

A. Merge đại vào, sửa sau

B. Email cho sếp hỏi tại sao

C. Xem Logs trên GitHub Actions, sửa lỗi ở máy local, rồi Push lại

D. Xóa bài test đang bị lỗi

Hands-on Lab

Thực hành xây dựng quy trình hoàn chỉnh

Lab Overview

Xây dựng một ứng dụng Java Console đơn giản:

1. Khởi tạo bằng Maven
2. Thêm thư viện commons-lang3 (để demo util) và junit
3. Cấu hình Logback
4. Viết Unit Test
5. Tích hợp Checkstyle
6. Đẩy lên GitHub và xem Action chạy

Step 1: Init Maven Project

- Sử dụng IDE (IntelliJ/Eclipse) hoặc Command Line:

```
mvn archetype:generate \
  -DgroupId=com.lab.devops \
  -DartifactId=demo-ci \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DinteractiveMode=false
```

Sau đó mở thư mục `demo-ci` bằng IDE

Step 2: Update POM.xml

- Thêm dependencies vào file ‘pom.xml’:

```
<dependencies>
    <!-- Logback Classic (bao gồm SLF4J) -->
    <dependency>
        <groupId>ch.qos.logback</groupId>
        <artifactId>logback-classic</artifactId>
        <version>1.4.11</version>
    </dependency>

    <!-- JUnit 5 -->
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.9.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Step 3: Java Code & Logging

- Edit ‘src/main/java/com/lab/devops/App.java’:

```
package com.lab.devops;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class App {
    private static final Logger logger = LoggerFactory.getLogger(App.class);

    public static void main(String[] args) {
        logger.info("Starting Application...");
        int result = add(5, 10);
        logger.info("Result is: {}", result);
    }

    public static int add(int a, int b) {
        return a + b;
    }
}
```

Step 4: Create Unit Test

- Create ‘src/test/java/com/lab/devops/AppTest.java’:

```
package com.lab.devops;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

public class AppTest {
    @Test
    public void testAdd() {
        assertEquals(15, App.add(5, 10));
    }

    @Test
    public void testAddNegative() {
        assertEquals(-5, App.add(5, -10));
    }
}
```

Step 5: Configure Checkstyle

- Thêm <plugin> vào ‘pom.xml’ phần ‘<build>’:

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-checkstyle-plugin</artifactId>
    <version>3.3.0</version>
    <configuration>
        <configLocation>google_checks.xml</configLocation>
        <failsOnError>true</failsOnError>
    </configuration>
    <executions>
        <execution>
            <phase>validate</phase>
            <goals><goal>check</goal></goals>
        </execution>
    </executions>
</plugin>
```

Step 6: Run Local & Fix Errors

- Thêm <plugin> vào ‘pom.xml’ phần ‘<build>’:

```
mvn clean install
```

Khả năng cao sẽ lỗi Checkstyle:

- Missing Javadoc
- Line length
- Indentation

Nhiệm vụ: Đọc lỗi và sửa file ‘App.java’ cho đến khi Build Success

Step 7: Push to GitHub & Action

- Thêm <plugin> vào ‘pom.xml’ phần ‘<build>’:

```
mvn clean install
```

1. Tạo repo mới trên GitHub
2. Push code lên
3. Tạo file `.github/workflows/maven.yml` (như mẫu Slide 39)
4. Vào Tab **Actions** để xem quy trình chạy

Thực hành

Level 1: Badge

Thêm Badge trạng thái build (Passing/Failing) vào file README.md.

Level 2: Coverage

Tích hợp plugin JaCoCo để đo độ bao phủ test (Test Coverage).

Yêu cầu build fail nếu coverage < 80%.

Tổng kết

- **Maven**: Giúp quản lý thư viện và quy trình build chuẩn
- **Logging**: Giúp quan sát ứng dụng tường minh hơn System.out
- **Checkstyle**: Giữ code sạch, dễ đọc, thống nhất
- **GitHub Actions**: Đảm bảo chất lượng code trước khi merge

Tài liệu tham khảo:

- Apache Maven Guides
- Logback Manual
- GitHub Actions Documentation